

算法复习重点

算法复习重点

- 1.summary
- 2.第一章
- 3.第二章
 - 二分搜索
 - 合并排序 稳定 最坏 $O(n \log n)$, 平均 $O(n \log n)$
 - 简单版
 - 改进版 (非递归算法)
 - 快速排序 最坏 $O(n^2)$ 平均 $O(n \log n)$
 - 线性时间选择 $O(n)$
 - 循环赛
 - 最接近点对问题 $O(n \log n)$
- 4.第三章
 - 最优子结构性性质
 - 重叠子问题——他和分治的不同点 (实际上很多和分治是一样的)
 - 动态规划算法设计步骤
 - 矩阵连乘——以最少的次数求出最终结果
 - 第一种办法: 自底向上
 - 第二种方法: 记录表方法——自顶向上
 - 最长公共子序列——最长公共子序列, 怎么找原问题和子问题!! 倒着找!
 - 扩展: 三角剖分
 - 最大子段和 $O(n)$
 - 0/1背包问题
- 5.第四章
 - 理解贪心算法概念
 - 掌握贪心算法基本要素
 - 最优子结构性性质
 - 贪心选择性质
 - 理解贪心和动态规划差异
 - 理解贪心算法的一般理论
 - 活动安排
 - 背包问题
 - 哈夫曼编码 很少考 太难了
 - 最小生成树
 - Prime 最近顶点 $O(n^2)$
 - Kruskal 最短边 $O(e \log e)$
 - 单源最短路径 $O(n^2)$
- 6.第五章
 - 解向量、解空间树、剪枝函数、剪枝后的解空间树、伪代码、复杂度
 - 理解回溯法的深度优先搜索策略
 - 掌握回溯法解题的算法框架
 - 轮船装载问题 $O(2^n)$
 - 作业调度问题 $O(n!)$
 - TSP旅行售货员问题 $O(n!)$
 - N皇后问题
 - 图的着色问题 $O(mn)$

1.summary

2.第一章

第1章的复习 (1)

1. 算法复杂性的概念

- 时间、空间复杂性
- 5种渐进复杂性定义
- $O, o, \Omega, \omega, \Theta$ 的概念
- ! 证明

$$f(n) = ?(g(n))$$

? : 5种渐近复杂性

- 注意: O 、 Ω 与 o 、 ω 在定义上的区别:
- 存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有
- 对于任何正常数 $c > 0$, 存在正数 $n_0 > 0$

$$f(n) = O(g(n)) \approx a \leq b; \text{ 渐近上界}$$

$$f(n) = \Omega(g(n)) \approx a \geq b; \text{ 渐近下界}$$

$$f(n) = \Theta(g(n)) \approx a = b; \text{ 紧渐近界}$$

$$f(n) = o(g(n)) \approx a < b; \text{ 非紧上界}$$

$$f(n) = \omega(g(n)) \approx a > b; \text{ 非紧下界}$$

2. 用特征方程解递归方程的通解

参考纸质版作业题, 就是考试题型
(时间复杂度 部分空间复杂度)
证明, 计算复杂度 (非递归, 递归)
非递归: 课件有一张表
递归: 解递归方程 (解特征方程)

下午 8:29 12月24日周四

80%

时间复杂性渐进阶分析的规则(最坏情况)

- 1). 赋值, 比较, 算术运算, 逻辑运算, 读写单个变量(常量)只需1单位时间
- 2). 执行条件语句 **if c then S1 else S2** 的时间为 $T_C + \max(T_{S1}, T_{S2})$.
- 3). 选择语句 **case A of a1: s1; a2: s2; ...; am: sm**
需要的时间为 $\max(T_{S1}, T_{S2}, \dots, T_{Sm})$.
- 4). 访问数组的单个分量或纪录的单个域需要一个单位时间.
- 5). 执行**for**循环语句的时间=执行循环体时间*循环次数.
- 6). **while c do s (repeat s until c)**语句时间= $(T_C + T_S) \times$ 循环次数.
- 7). 用**goto**从循环体内跳到循环体末或循环后面的语句时, 不需额外时间
- 8). 过程或函数调用语句
对非递归调用, 根据调用层次由里向外用规则1-7进行分析;
对递归调用, 可建立关于 **$T(n)$** 的递归方程, 求解该方程得到 **$T(n)$** .

套用公式法

大小为 n 的原问题分成若干个大小为 n/b 的子问题，其中 a 个子问题需要求解，而 cn^k 是合并各个子问题的解需要的工作量。

$$T(n) = \begin{cases} c & n = 1 \\ aT(n/b) + cn^k & n > 1 \end{cases}$$

$$T(n) = \begin{cases} O(n^{\log_b a}) & a > b^k \\ O(n^k \log_b n) & a = b^k \\ O(n^k) & a < b^k \end{cases}$$

$$X_n = 5^n + (-2)^n$$

Homework 1
Proof and compute.

纸质
2周后(第四周)

- 证明关系
 - $3n^2 + 4n^3 = O(n^3)$
- 求解线性递归关系:
 - $\begin{cases} x_{n+1} = 3x_n + 10x_{n-1}, n \geq 1 \\ x_0 = 2, x_1 = 3 \end{cases}$

特征根 $q_1, q_2, \dots, q_{r-1}$

$$f(n) = C_1 q_1^n + \dots + C_{r-1} q_{r-1}^{n-1} q_1^{r-1}$$

$$\begin{aligned} X^{n+1} &= 3X^n + 10X^{n-1} \\ X^2 &= 3X + 10 \\ X^2 - 3X - 10 &= 0 \\ (X-5)(X+2) &= 0 \\ X_1 &= 5 \quad X_2 = -2 \\ X^n &= C_1 5^n + C_2 (-2)^n \\ X_0 &= C_1 + C_2 = 2 \Rightarrow C_1 = 1 \\ X_1 &= 5C_1 - 2C_2 = 3 \Rightarrow C_2 = 1 \end{aligned}$$

3.第二章

第2章 分治法

1. 理解递归的概念和原理
2. 掌握设计有效的分治算法的方法
3. 利用范例学习：
 - ① 二分搜索
 - ② 快速排序
 - ③ 合并排序
 - ④ 线性时间选择
4. 要求：

能够流利写出范例中的算法伪代码，并能分析算法的时间复杂度。

二分搜索源问题分成两个子问题，只在一个子问题求解，的复杂性， $O(\log n)$ ，递归方程，尽量等分，中位数的选取方法。

合并排序的思想，原问题化为子问题，非递归算法。

⊕ 循环赛

⊕ 最近点对问题（二维）重点关注在一定的划分区间里，在右半寻找6个点，要知道是哪6个点，伪代码要写

分析算法在各种情况下的时间复杂度

快排的思想不稳定，排好序时性能很差。

伪代码用c++

还会问一下拓展问题

合并排序（渐进意义上的 $O(n \log n)$ ）快速排序（不退化 $O(n \log n)$ 那种情况下发生退化，解决方法：不再选取第一个，随机选取，改写partition，去避免退化问题）合并排序某种意义上更耗时间

稳定 非稳定

求第k小，借鉴了partition

线性时间选择 $O(n)$ 倍，伪代码也要会

分治法的复杂性分析

一个分治法将规模为 n 的问题分成 k 个规模为 n/m 的子问题去解。设分解阈值 $n_0=1$ ，且ad hoc解规模为1的问题耗费1个单位时间。再设将原问题分解为 k 个子问题以及用merge将 k 个子问题的解合并为原问题的解需用 $f(n)$ 个单位时间。用 $T(n)$ 表示该分治法解规模为 $|P|=n$ 的问题所需的计算时间，则有：

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

通过迭代法求得方程的解： $T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(n/m^j)$

注意：递归方程及其解只给出 n 等于 m 的方幂时 $T(n)$ 的值，但是如果认为 $T(n)$ 足够平滑，那么由 n 等于 m 的方幂时 $T(n)$ 的值可以估计 $T(n)$ 的增长速度。通常假定 $T(n)$ 是单调上升的，从而当 $m^i \leq n < m^{i+1}$ 时， $T(m^i) \leq T(n) < T(m^{i+1})$ 。

二分搜索

//分治法：二分搜索

```
public static int binarySearch(int [] a, int x, int n)
{
    // 在 a[0] <= a[1] <= ... <= a[n-1] 中搜索 x
    // 找到x时返回其在数组中的位置，否则返回-1
    int left = 0; int right = n - 1;
    while (left <= right)
    {
        int middle = (left + right)/2;
        if (x == a[middle]) return middle;
        if (x > a[middle]) left = middle + 1; //在右半部查找
        else right = middle - 1;           //在左半部查找
    }
    return -1; // 未找到x
}
```

$$T(n) = 1 \cdot T(n/2) + \dots$$

算法复杂度分析：

每执行一次算法的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了 $O(\log n)$ 次。循环体内运算需要 $O(1)$ 时间，因此整个算法在最坏情况下的计算时间复杂性为 $O(\log n)$ 。

只在其中一个问题进行查找

$$O(n) \rightarrow O(\log n)$$

合并排序 稳定 最坏 $O(n \log n)$ ，平均 $O(n \log n)$

某种意义上更耗时间

简单版

MERGE-SORT

```
MERGE-SORT(A, p, r)
{
    if p < r
    {
        q = ((p+r)/2);
        MERGE-SORT(A, p, q);
        MERGE-SORT(A, q+1, r);
        MERGE(A, p, q, r);
    }
}
```

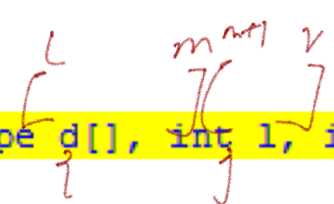
MERGE(A, p, q, r)

```
{
1:  n1 = q-p+1;          //length(L)
2:  n2 = r-q;            //length(R)
3:  creat_arrays L[1...n1+1],R[1...n2+1];
4:  for i=1:n1
5:      do L[i] = A[p+i-1];
6:  for j=1:n2
7:      do R[j] = A[q+j];
8:  L[n1+1] = MAX;
9:  R[n2+1] = MAX;
10: i = 1;
11: j = 1;
12: for k = p:r
13: {
14:     if L[i] <= R[j]
15:     {
16:         A[k] = L[i];
17:         i = i+1;
18:     }
19:     else
20:     {
21:         A[k] = R[j];
22:         j = j+1;
23:     }
24: }
```

MERGE(A, p, q, r)

改进版 (非递归算法)

//合并算法



```
void Merge( Type c[], Type d[], int l, int m, int r)
{
    int i = l, j = m+1, k = 1;
    while(( i <=m ) && ( j <= r ))
    {
        if ( c[i] <= c [j] ) d[k++] = c[i++];
        else d[k++] = c [j++];
    }
    if ( i > m ) 左列耗尽
    {
        for (int q = j; q <= r; q ++ )
            d[k++] = c[q];
    }
    else for ( int q =i; q <= m; q++ )
        d[k++] = c[q];
}
```

```
//mergepass
```

```
void MergePass ( Type x[], Type y[], int s, int n )
{
    int i = 0;
    while ( i <= n - 2 * s )
    {
        //合并大小为s的相邻2段子数组
        Merge( x, y, i, i+s-1, i+2*s-1);
        i = i + 2 * s;
    }
    //剩下的元素个数少于2s (n不是2的倍数)
    if ( i + s < n ) Merge ( x, y, i, i+s-1, n-1);
    else for (int j = i; j <= n-1; j++)
        y[j] = x[j];
}
```

```
//合并排序算法
```

```
void MergeSort ( Type a[], int n )
{
    Type *b = new Type [n];
    int s = 1;
    while ( s < n )
    {
        MergePass( a, b, s, n);
        s += s;
        MergePass( b, a, s, n);
        s += s;
    }
}
```

快速排序 最坏 $O(n^2)$ 平均 $O(n \log n)$

排序算法的思想非常简单，在待排序的数列中，我们首先要找一个数字作为基准数（这只是个专用名词）。为了方便，我们一般选择第 1 个数字作为基准数（其实选择第几个并没有关系）。接下来我们需要把这个待排序的数列中小于基准数的元素移动到待排序的数列的左边，把大于基准数的元素移动到待排序的数列的右边。这时，左右两个分区的元素就相对有序了；接着把两个分区的元素分别按照上面两

种方法继续对每个分区找出基准数，然后移动，直到各个分区只有一个数时为止。

```
private void quickSort(int[] src, int begin, int end) {
    if (begin < end) {
        int key = src[begin];
        int i = begin;
        int j = end;
        while (i < j) {
            while (i < j && src[j] > key) {
                j--;
            }
            if (i < j) {
                src[i] = src[j];
                i++;
            }
            while (i < j && src[i] < key) {
                i++;
            }
            if (i < j) {
                src[j] = src[i];
                j--;
            }
        }
        src[i] = key;
        quickSort(src, begin, i - 1);
        quickSort(src, i + 1, end);
    }
}
```

QUICKSORT(A, p, r)

if $p < r$

then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT($A, p, q-1$)

 QUICKSORT($A, q+1, r$)

Initial call: QUICKSORT($A, 1, n$)

 ▷ pivot = $A[p]$

do if $A[j] \leq x$

then $i \leftarrow i + 1$

 exchange $A[i] \leftrightarrow A[j]$

PARTITION(A, p, q) ▷ $A[p \dots q]$

$x \leftarrow A[p]$ ▷ pivot = $A[p]$

$i \leftarrow p$

for $j \leftarrow p + 1$ **to** q

do if $A[j] \leq x$

then $i \leftarrow i + 1$

 exchange $A[i] \leftrightarrow A[j]$

 exchange $A[p] \leftrightarrow A[i]$

return i

Running time
= $O(n)$ for n
elements.

不稳定，不退化 $O(n \log n)$

下面我们讨论一下发生退化的情况，也就是我们选取的key，也就是partition，他一直是最大值或者最小值，那这时快排就变成了一个冒泡排序，就变成了 $O(n^2)$ (比较了 $n(n-1)/2$)

解决方法：不再选取第一个，随机选取，改写partition，去避免退化问题


```
private static int randomizedPartition (int p, int r)
{
    int i = random(p,r);
    MyMath.swap(a, i, p);
    return partition (p, r);
}
```

扩展：求第k小

一、先排序，然后直接取。最坏情况下至少 $O(n \log n)$

二、极值情况：数组中只有一个元素；利用二分法进行划分，分成两个数组，左半部和右半部，左半部小于右半部。统计出左半部的元素个数 i ($k \leq i$) k 一定出现在左半部。否则， k 在右半部，则问题变成了在右半部递归查找第 $k-i$ 小元素。最坏 $O(n^2)$ ，但随机partition可以为 $O(n)$ 。

线性时间选择 $O(n)$

如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ϵ 倍($0 < \epsilon < 1$ 是某个正常数)，那么就可以在最坏情况下用 $O(n)$ 时间完成选择任务，这是线性时间选择问题。

Pivot/划分基准的选择：

1. 将 n 个输入元素划分成 $n/5$ 个组，每组5个元素，只可能有一个组不是5个元素。用任何一种排序算法，将每组中的元素排好序，并取出每组的中位数，共 $n/5$ 个。

2. 递归调用select来找出这 $n/5$ 个元素的中位数。如果 $n/5$ 是偶数，就找它的2个中位数中较大的一个。

3. 以这个元素作为划分基准。

```

Type Select(Type a[], int p, int r, int k)
{
    if (r-p<75) {
        用某个简单排序算法对数组a[p:r]排序;
        return a[p+k-1];
    },
    for ( int i = 0; i<=(r-p-4)/5; i++ )
        将a[p+5*i]至a[p+5*i+4]的第3小元素
        与a[p+i]交换位置;
    //找中位数的中位数, r-p-4即上面所说的n-5
    Type x = Select(a, p, p+(r-p-4)/5, (r-p-4)/10);
    int i=Partition(a,p,r,x),
    j=i-p+1;
    if (k<=j) return Select(a,p,i,k);
    else return Select(a,i+1,r,k-j);
}

```

循环赛

$n=2^k$ 个运动员, 彼此都得比一次, 一个选手一天比一次, 比 $n-1$ 天

按分治策略, 将所有的选手分为两半, n 个选手的比赛日程表就可以通过为 $n/2$ 个选手设计的比赛日程表来决定。递归地用对选手进行分割, 直到只剩下2个选手时, 比赛日程表的制定就变得很简单。这时只要让这2个选手进行比赛就可以了。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

具体参见循环赛实验报告

最接近点对问题 $O(n \log n)$

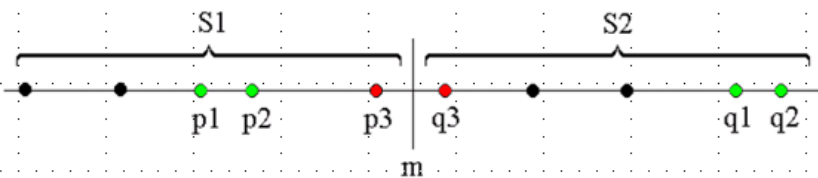
重点关注在一定的划分区间里, 在右半寻找6个点, 要知道是哪6个点, 伪代码要写

对于一维

一、算出每一个点和其他点的距离, 然后比较。 $O(n^2)$

二、在直线上, 最近点对问题可以通过一次排序和一维扫面来解决。 $O(n \log n)$

如何找 p_3, q_3 ?



- ◆如果S的最接近点对是 $\{p_3, q_3\}$ ，即 $|p_3 - q_3| < d$ ，则 p_3 和 q_3 两者与 m 的距离不超过 d ，即 $p_3 \in (m-d, m]$ ， $q_3 \in (m, m+d]$ 。
- ◆由于在 S_1 中，每个长度为 d 的半闭区间至多包含一个点（否则必有两点距离小于 d ），并且 m 是 S_1 和 S_2 的分割点，因此 $(m-d, m]$ 中至多包含S中的一个点。由图可以看出，如果 $(m-d, m]$ 中有S中的点，则 p_3 就是 S_1 中最大点。
- ◆同理， q_3 就是 S_2 中的最小点。

分治法的思路:

- ① 划分成子规模点集 S_1 和 S_2
- ② 找到 S_1 和 S_2 的最小距离 d 。
- ③ 合并 S_1 和 S_2 ：并利用 d 在 S_1 的 $(m-d, m]$ 和 S_2 的 $(m, m+d]$ 中找最大点和最小点，即 p_3 和 q_3 。选出最小距离，完成合并

分治法的算法复杂性:

- ① 划分成子规模点集 S_1 和 S_2
- ② 我们用线性时间就能找到区间 $(m-d, m]$ 和 $(m, m+d]$ 中所有点，即 p_3 和 q_3 。从而我们用线性时间就可以将 S_1 的解和 S_2 的解合并成为S的解。

划分也是线性的。

合并：找子集中的最大最小值，是一个 $n-1$ 的比较过程，是线性的。如果不是这样，则对于 S_1 中的每一个点都要找 $n/2$ 个，一共是 $n/2 * n/2$

如何选择 m ——一般是中位数 $O(\log n)$

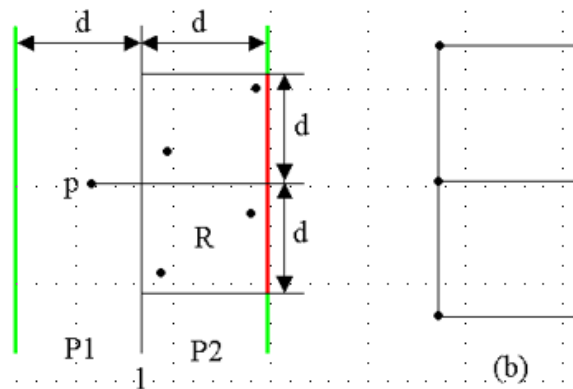
//一维

bool Cpair1(s, d)

```
{
    n = |s|;
    if (n < 2) {d为无穷大; return false;}
    m = s中各个坐标的中位数;
    //构造S1 和 S2
    //S1 = { x < m } s2 = { x > m }
    Cpair1( s1, d1 );
    Cpair1( s2, d2 );
    p = max (s1);
    q = min (s2);
    d = min( d1, d2, q-p);
    return true;
}
```

对于二维

- ◆考虑P1中任意一点p，它若与P2中的点q构成最接近点对的候选者，则必有 $\text{distance}(p, q) < d$ 。满足这个条件的P2中的点一定落在一个 $d \times 2d$ 的矩形R中
- ◆由d的意义可知，P2中任何2个S中的点的距离都不小于d。由此可以推出矩形R中最多只有6个S中的点。
- ◆因此，在分治法的合并步骤中最多只需要检查 $n/2 \times 6 = 3n$ 个候选者



double cpair2(S)	4、设P1是S1中距垂直分割线l的距离在 d_m 之内的所有点组成的集合；
{	P2是S2中距分割线l的距离在 d_m 之内所有点组成的集合；
n= S ;	将P1和P2中点依其y坐标值排序；
if (n < 2) return false;	并设X和Y是相应的已排好序的点列；
1、m=S中各点x间坐标的中位数;	5、通过扫描X以及对于X中每个点检查Y中与其距离在 d_m 之内的所有点(最多6个)可以完成合并；
构造S1和S2;	当X中的扫描指针逐次向上移动时，Y中的扫描指针可在宽为 $2d_m$ 的区间内移动；
//S1={p ∈ S x(p) ≤ m},	设d1是按这种扫描方式找到的点对间的最小距离；
//S2={p ∈ S x(p) > m}	6、 $d = \min(d_m, d1)$;
2、d1=cpair2(S1);	return d;
d2=cpair2(S2);	}
3、 $d_m = \min(d1, d2)$;	

第二步时间为 $T(n/2)$ ，第四步和第五步的扫描放在一起只要 O_n （因为已经排好序了预排序）

$$T_n = 2T_{n/2} + O_n$$

4.第三章

答题的时候一定要写递推式!!! 包括其他题，注意路径！代码10分（最优解5分，最优解构造5分），分析算法的复杂度分析5分，递推式子5分等等都要算分。

一定在考前要写一遍代码。知道所有问题的最优子结构和证明。

最优子结构性质

原问题的最优解包含了其子问题的最优解，即原问题可以由子问题的最优解组合而成，这就使得问题可以拆分成若干个子问题。

重叠子问题——他和分治的不同点（实际上很多和分治是一样的）

每次产生的子问题并不总是新问题，有些子问题被反复计算多次。

动态规划算法设计步骤

找出最优解的性质，并刻画其结构特征。
递归地定义最优值。
以自底向上的方式计算出最优值。
根据计算最优值时得到的信息，构造最优解。

以下范例都要伪代码 实现求最优解和构造最优解 分析时间复杂度

矩阵连乘——以最少的次数求出最终结果

是聪明的全遍历。自底向上的求解（也就是从最小问题到最大问题）

这里涉及到了贪心算法——得不到全局最优解

选计算量小的优先算；选矩阵维数大的优先算

◆思考四：分治法

设 n 个矩阵连乘积 $A_1 A_2 \cdots A_n$ 的计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵

链断开， $1 \leq k < n$ ，则其相应的完全加括号方式为 $((A_1 \cdots A_k)(A_{k+1} \cdots A_n))$ 。

由于完全加括号方式是一个递归定义，因此可以递归地计算 $A[1:k]$ 和

$A[k+1:n]$ ，然后将计算结果相乘得到 $A[1:n]$ 。据此可设计如下递归算法 **recurMatrixChain**：

```
int recurMatrixChain(int i, int j)
{
    if (i == j) return 0;
    int u = recurMatrixChain(i, i) + recurMatrixChain(i+1, j) + p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k = i+1; k < j; k++) {
        int t = recurMatrixChain(i, k) + recurMatrixChain(k+1, j) + p[i-1]*p[k]*p[j];
        if (t < u) {
            u = t;
            s[i][j] = k;
        }
    }
    return u;
}
```

但分治法效率很低，因为很多子问题被重复计算。 $O(2^n)$

第一种办法：自底向上

下面开始动态规划：

1. 分析最优解的结构

- 特征：计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的。
- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法求解的显著特征。
- 如何证明？——反证法

2. 建立递归关系

- 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少数乘次数 $m[i,j]$, 则原问题的最优值为 $m[1,n]$
- 当 $i=j$ 时, $A[i:j]=A_i$, 因此, $m[i,i]=0$, $i=1,2,\dots,n$
- 当 $i < j$ 时,

$$m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$$

这里 A_i 的维数为 $p_{i-1} \times p_i$

- 由最优子结构性质, 可以递归地定义 $m[i,j]$ 为:

$$m[i,j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

```
void MatrixChain(int *p, int n, int **m, int **s)
```

```
{
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int r = 2; r <= n; r++) // r代表了对角线, 如图a
        for (int i = 1; i <= n - r + 1; i++) { // 代表了对每条对角线的数值
            int j = i + r - 1;
            m[i][j] = m[i+1][j] + p[i-1]*p[i]*p[j];
            s[i][j] = i;
            for (int k = i+1; k < j; k++) {
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k; }
            }
        }
}
```

A1
30×35

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][3] + p_1p_2p_3 \\ m[2][3] + m[4][4] + p_1p_3p_4 \\ m[2][4] + m[5][5] + p_1p_4p_5 \end{cases}$$

时间 $O(n^3)$, 空间 $O(n^2)$ 。然后借助这个 $s[i,j]$ 去给 $A[i,j]$ 进行加括号

因此, 从 $s[1][n]$ 记录的信息可知计算 $A[1:n]$ 的最优加括号方式为 $(A[1:s[1][n]])(A[s[1][n]+1:n])$ 。

而 $A[1:s[1][n]]$ 的最优加括号方式为

$(A[1:s[1][s[1][n]]])(A[s[1][s[1][n]]+1:s[1][s[1][n]]])$ 。同理可以确定 $A[s[1][n]+1:n]$ 的最优加括号方式在 $s[s[1][n]+1][n]$ 处断开,, 照此递推下去, 最终可以确定 $A[1:n]$ 的最优

```
void traceback (int i, int j, int ** s)
{
    if (i == j) return;
    traceback(s, i, s[i][j]);
    traceback(s, s[i][j] + 1, j);
    cout<<"Multiply A "<< i << ", " << s[i][j];
    cout<<" and A " << (s[i][j] + 1) << ", " << j << endl;
}
```

调用`traceback(1,n,s)`即可

第二种方法: 记录表方法——自顶向上

备忘录方法的控制结构与直接递归方法的控制结构相同, 区别在于备忘录方法为每个解过的子问题建立了备忘录以备需要时查看, 避免了相同子问题的重复求解。

```
int LookupChain(int i, int j)
{
    if (m[i][j] > 0) return m[i][j]; //如果已经计算过, 直接返回即可
    if (i == j) return 0;
    int u = LookupChain(i, i) + LookupChain(i+1, j) + p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k = i+1; k < j; k++) {
        int t = LookupChain(i, k) + LookupChain(k+1, j) + p[i-1]*p[k]*p[j];
        if (t < u) { u = t; s[i][j] = k; }
    }
    m[i][j] = u;
    return u;
}
```

动态规划法: 自底向上
备忘录方法: 自顶向下

矩阵连乘及最优三角剖分: 递推式子, 伪代码, 最优解怎样构造最优解, 算法复杂度

最长公共子序列——最长公共子序列, 怎么找原问题和子问题!! 倒着找!

最优子结构

1. 最长公共子序列的结构

设序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z=\{z_1, z_2, \dots, z_k\}$, 则

(1) 若 $x_m=y_n$, 则 $z_k=x_m=y_n$, 且 Z_{k-1} 是 x_{m-1} 和 y_{n-1} 的最长公共子序列。

(2) 若 $x_m \neq y_n$ 且 $z_k \neq x_m$, 则 Z 是 x_{m-1} 和 Y 的最长公共子序列。

(3) 若 $x_m \neq y_n$ 且 $z_k \neq y_n$, 则 Z 是 X 和 y_{n-1} 的最长公共子序列。

由此可见, 2个序列的最长公共子序列包含了这2个序列的前缀的最长公共子序列。因此, 最长公共子序列问题具有最优子结构性质。

递归结构

2. 子问题的递归结构

由最长公共子序列问题的最优子结构性质建立子问题最优值的递归关系。

用 $c[i][j]$ 记录序列 X_i 和 Y_j 的最长公共子序列的长度。其中,

$X_i=\{x_1, x_2, \dots, x_i\}$; $Y_j=\{y_1, y_2, \dots, y_j\}$ 。

当 $i=0$ 或 $j=0$ 时, 空序列是 X_i 和 Y_j 的最长公共子序列。故此时 $C[i][j]=0$ 。其它情况下, 由最优子结构性质可建立递归关系如下:

$$c[i][j] = \begin{cases} 0 & i=0, j=0 \\ c[i-1][j-1]+1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

```
void LCSLength(int m, int n, char *x,
char *y, int **c, int **b)
```

```
{
    int i, j;
    for (i = 1; i <= m; i++) c[i][0] = 0;
    for (i = 1; i <= n; i++) c[0][i] = 0;
    for (i = 1; i <= m; i++)
        for (j = 1; j <= n; j++) {
            if (x[i]==y[j]) {
                c[i][j]=c[i-1][j-1]+1; b[i][j]=1;}
            else if (c[i-1][j]>c[i][j-1]) {
                c[i][j]=c[i-1][j]; b[i][j]=2;}
            else { c[i][j]=c[i][j-1]; b[i][j]=3;}
        }
}
```

构造最长公共子序列

```
void LCS(int i, int j, char *x, int **b)
```

```
{
    if (i==0 || j==0) return;
    if (b[i][j]==1){ LCS(i-1, j-1, x, b); cout<<x[i]; }
    else if (b[i][j]==2) LCS(i-1, j, x, b);
    else LCS(i, j-1, x, b);
}
```

$O(mn)$ 。但其实可以将数组 b 省略。而且可以看到我们只用到数组 c 的第 i 行和第 $i-1$ 行, 用两行就行。

可以减少算法复杂度到 $O(\min\{m,n\})$

最长公共子序列及其推广，最长上升/递减子序列，最短回文串

利用最优子结构写出递推式子3-5分，伪代码5-7分

扩展：三角剖分

最优三角剖分的递归结构

- 定义 $t[i][j]$, $1 \leq i < j \leq n$ 为凸子多边形 $\{v_{i-1}, v_i, \dots, v_j\}$ 的最优三角剖分所对应的权函数值，即其最优值。为方便起见，设退化的多边形 $\{v_{i-1}, v_i\}$ 具有权值0。据此定义，要计算的凸 $(n+1)$ 边形 P 的最优权值为 $t[1][n]$ 。
- $t[i][j]$ 的值可以利用最优子结构性质递归地计算。当 $j-i \geq 1$ 时，凸子多边形至少有3个顶点。由最优子结构性质， $t[i][j]$ 的值应为 $t[i][k]$ 的值加上 $t[k+1][j]$ 的值，再加上三角形 $v_{i-1}v_kv_j$ 的权值，其中 $i \leq k \leq j-1$ 。由于在计算时还不知道 k 的确切位置，而 k 的所有可能位置只有 $j-i$ 个，因此可以在这 $j-i$ 个位置中选出使 $t[i][j]$ 值达到最小的位置。由此， $t[i][j]$ 可递归地定义为：

$$t[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{t[i][k] + t[k+1][j] + w(v_{i-1}v_kv_j)\} & i < j \end{cases}$$

最大子段和 On

分治法 $O(n \log n)$ ——联系最接近点对问题

```
int MaxSubSum(int *a, int left, int right)
{
    int sum = 0;
    if (left == right) sum = a[left] > 0 ? a[left] : 0;
    else{
        int center = (left + right) / 2;
        int leftsum = MaxSubSum(a, left, center);
        int rightsum = MaxSubSum(a, center+1, right);

        int s1 = 0;
        int lefts = 0;
        for (int i = center; i >= left; i--)
        {
            lefts += a[i];
            if (lefts > s1) s1 = lefts;
        }

        int s2 = 0;
        int rights = 0;
        for (int i = center + 1; i <= right; i++)
        {
            rights += a[i];
            if (rights > s2) s2 = rights;
        }

        sum = s1 + s2;
        if (sum < leftsum) sum = leftsum;
        if (sum < rightsum) sum = rightsum;
    }
    return sum;
}
```

下面开始动态规划——我们只枚举开始点或者只枚举结束点

记 $b[j] = \max_{1 \leq i \leq j} \{ \sum_{k=i}^j a[k] \}$, $1 \leq j \leq n$, 则所求的最大子段和为

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} \max_{1 \leq i \leq j} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} b[j]$$

当 $b[j-1] > 0$ 时 $b[j] = b[j-1] + a[j]$, 否则 $b[j] = a[j]$ 。由此可得计算 $b[j]$ 的动态规划递归式

$$b[j] = \max\{b[j-1] + a[j], a[j]\}, \quad 1 \leq j \leq n$$

```
int maxSum(int n, int * a)
{
    int sum=0, b=0;
    for (int i=1; i<=n; i++) {
        if (b>0) b+=a[i];
        else b=a[i];
        if (b>sum) sum=b;
    }
    return sum;
}
```

On时间复杂度

最大字段和及其推广：最大字段和的值和该最大字段

倒着遍历，记录所有子段

另外解法

令 $L[i]$ 表示 $1 \sim i$ 的最大字段和, $R[i]$ 表示 $i+1 \sim n$ 的最大字段和, ($R[i]$ 是倒着算的)

则 $Ans = \max\{L[i] + R[i]\} (1 \leq i \leq n - 1)$

假如我现在知道以下标 i 结束的数组和 (注意是数组和, 就是下标 i 与之前的全部元素加一起), 如上面的和分别为: 1, -4, -1, 3, 1, -2, 8。下标 i 结束的数组和记为 $sum(i)$, 如果利用 $sum(i)$ 减去下标 i 以前的最小和 $sum(j)$, 这里 ($j < i$), 即 $sum(i) - sum(j)$ 就是以 i 为结尾的最大子数组和。即为解法三中的 $dp[i]$ 的值。

分析：

可以联想到求最大子段和的方法，此题就是以此为基础的。

先从左向右做一遍求最大子段和的DP，得到数组dp1[];

再从右向左做一遍求最大子段和的DP，得到数组dp2[]。

然后更新一下这两个数组，更新方法为：

对于数组dp1[]，若 $dp1[i-1] > dp1[i]$, $dp[i] = dp1[i-1]$, 否则不变;

对于数组dp2[]，若 $dp2[i+1] > dp2[i]$, $dp[i] = dp2[i+1]$, 否则不变。

以样例为例：

1 -1 2 2 3 -3 4 -4 5 -5

得到

dp1----->1 1 2 4 7 7 8 8 9 9

dp2----->9 9 9 7 5 5 5 5 5 -5

最后枚举1~n-1，求 $dp1[i] + dp2[i+1]$ 的最大值，即为答案。

```
int getmax3_N(int a[],int n)↓
{↓
    int sum=0,sumMin=0,Max=a[0];↓
    for(int i=0;i<n;i++)↓
    {↓
        if(sumMin>sum)↓
            sumMin=sum;↓
        sum=sum+a[i];↓
        if(Max<sum-sumMin)↓
            Max=sum-sumMin;↓
    }↓
    return Max;↓
}↓
```


下面解决如何记录起始点与结束点下标

在动态规划的时候最好从后向前递推，这样 $dp[i]$ 表示的值就是以下标 i 为开始的最大子数组的值，那么当 $dp[i]$ 与 $dp[j]$ 相同时我们选取 i,j 中较小的下标作为起点。

```
if(dp[i+1]>0)
{
    dp[i]=dp[i+1]+a[i];
    tmp[i]=tmp[i+1];
}
else
{
    dp[i]=a[i];
    tmp[i]=i;
}
```

- 这里 tmp 变量记录子数组的结束点，当 $dp[i+1]==0$ 时，我们要选取 i 作为此时的子数组结束点。如上面的例子2,3,-1,1；当我们计算 $dp[1]$ 时，它的结束点就是1，而不是 $dp[2]$ 的结束点3。

最终最大的 $dp[i]$ 代表最大子数组的和，最小的 i 代表子数组的起始点， $tmp[i]$ 代表子数组的结束点。

```
void getMax(int a[],int n)
{
    int dp,tmp;
    dp=Max=a[n-1];
    tmp=n-1;
    B=n-1;
    E=n-1;
    for(int i=n-2;i>=0;i--)
    {
        if(dp>0)
            dp=dp+a[i];
        else
        {
            dp=a[i];
            tmp=i;
        }
        if(Max<=dp)
        {
            B=i;
            E=tmp;
            Max=dp; } } }
```

int B,E,Max;//B为子数组的起始点下标,E为子数组的结束点下标

//Max为最大子数组的和。
//getMax求数组a(元素个数为n,以下标0作为起始点)的最大子数组，外加起点下标与//结束点下标。

由于 $dp[i]$ 只与 $dp[i+1]$ 有关系， $tmp[i]$ 只与 $tmp[i+1]$ 有关系，所以我们可以省略数组，改为变量 dp,tmp 。

推广：

求两个最大字段和的和值最大问题，用 bj 数组。当前面为负，就没有意义。只有大于0才加

设:

$dp1[i]$ 代表以下标 i 结束的最大子数组的值,

$max1[i]$ 代表 $dp1[0] \sim dp1[i]$ 最大的那个值;

设

$dp2[i]$ 代表以下标 i 开始的最大子数组的值,

$max2[i]$ 代表 $dp1[i] \sim dp1[n-1]$ 中最大的那个值。

例如, $max1[6]$ 就是 $dp[6]$ 即 $\{2,2,3,-3,4\}$; $max2[7]$ 就是 $dp[8]$ 即 $\{5\}$,
所以上面那个例子求出的解就是 $max1[6]+max2[7]$ 。而为何是 $max1[6]+max2[7]$, 是因为在 $max1[i]+max2[i+1](i=0 \sim n-2)$ 中, 它最大。我们只需要枚举这两段的分界点 i , $0 \leq i \leq n-2$, 就可以了。

最后得出方法:

- 利用标准动态规划求最大子子段和的方法, 求出 $dp1[]$ 、 $dp2[]$ 、 $max1[]$ 、 $max2[]$;
- 枚举出 $max1[i]+max2[i+1](i=0 \sim n-2)$ 中, 最大的那个数, 就是结果。

```
int max1[MAXN], max2[MAXN];
int getMax(int a[], int n)
{
    int Max, dp;
    max1[0] = dp = a[0];
    for(int i = 1; i < n; i++)
    {
        if(dp < 0)
            dp = a[i];
        else
            dp = dp + a[i];
        if(max1[i-1] < dp)
            max1[i] = dp;
        else
            max1[i] = max1[i-1];
    }
    max2[n-1] = dp = a[n-1];
    for(int i = n-2; i >= 0; i--)
    {
        if(dp < 0)
            dp = a[i];
        else
            dp = dp + a[i];
        if(max2[i+1] < dp)
            max2[i] = dp;
        else
            max2[i] = max2[i+1];
    }
    Max = max1[0] + max2[1];
    for(int i = 1; i < n-1; i++)
        if(Max < max1[i] + max2[i+1])
            Max = max1[i] + max2[i+1];
    return Max;
}
```

拓展为循环数组, 也就是成环

加入限制条件 m , m 个

解题思路： 设 $b(i, j)$ 表示数组 a 的前 j 项中 i 个子段和的最大值，且第 i 个子段含 $a[j]$ ($1 \leq i \leq m, 1 \leq j \leq n$)。则所求的最优值显然为

$$\max_{m \leq j \leq n} b(m, j)$$

与最大子段和问题类似，计算 $b(i, j)$ 的递归式为

$$b(i, j) = \max \{b(i, j-1) + a[j], \max_{i-1 \leq t < j} b(i-1, t) + a[j]\} \quad (1 \leq i \leq m, 1 \leq j \leq n)$$

初始时， $b(0, j)=0, (1 \leq j \leq n); b(i, 0)=0, (1 \leq i \leq m)$ 。

将问题拓展到二维，求一个矩阵的最大子矩阵的和

(1) 最大子矩阵和问题

给定一个 m 行 n 列的整数矩阵 a ，试求矩阵 a 的一个子矩阵，使其各元素之和为最大。

$$\text{记 } s(i_1, i_2, j_1, j_2) = \sum_{i=i_1}^{i_2} \sum_{j=j_1}^{j_2} a[i][j]$$

最大子矩阵和最问题的最优值为

$$\max_{\substack{1 \leq i_1 \leq i_2 \leq m \\ 1 \leq j_1 \leq j_2 \leq n}} s(i_1, i_2, j_1, j_2)$$

时间复杂性：

由于解最大子段和最问题的动态规划算法需要时间 $O(n)$ ，故算法的双重for循环需要计算时间 $O(m^2n)$ 。

(贪心法解决最大字段和，分析为什么没有达到最优解)

0/1背包问题

有可能要求用贪心法

0-1背包问题

设所给0-1背包问题的子问题

$$\max \sum_{k=i}^n v_k x_k \quad \begin{cases} \sum_{k=i}^n w_k x_k \leq j \\ x_k \in \{0,1\}, i \leq k \leq n \end{cases}$$

的最优值为 $m(i, j)$, 即 $m(i, j)$ 是背包容量为 j , 可选择物品为 $i, i+1, \dots, n$ 时0-1背包问题的最优值。由0-1背包问题的最优子结构性质, 可以建立计算 $m(i, j)$ 的递归式如下。

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$
$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

$O(nc)$

算法改进, 会有跳跃点 $O(\min\{nc, 2n\})$

对每一个确定的 $i(1 \leq i \leq n)$, 用一个表 $p[i]$ 存储函数 $m(i, j)$ 的全部跳跃点。表 $p[i]$ 可依计算 $m(i, j)$ 的递归式递归地由表 $p[i+1]$ 计算, 初始时 $p[n+1] = \{(0, 0)\}$ 。

- 函数 $m(i, j)$ 是由函数 $m(i+1, j)$ 与函数 $m(i+1, j-w_i) + v_i$ 作 $\max$ 运算得到的。因此, 函数 $m(i, j)$ 的全部跳跃点包含于函数 $m(i+1, j)$ 的跳跃点集 $p[i+1]$ 与函数 $m(i+1, j-w_i) + v_i$ 的跳跃点集 $q[i+1]$ 的**并集**中。易知, $(s, t) \in q[i+1]$ 当且仅当 $w_i \leq s < c$ 且 $(s-w_i, t-v_i) \in p[i+1]$ 。因此, 容易由 $p[i+1]$ 确定跳跃点集 $q[i+1]$ 如下

$$q[i+1] = p[i+1] \oplus (w_i, v_i) = \{(j+w_i, m(i, j)+v_i) \mid (j, m(i, j)) \in p[i+1]\}$$

- 另一方面, 设 (a, b) 和 (c, d) 是 $p[i+1] \cup q[i+1]$ 中的2个跳跃点, 则当 $c \geq a$ 且 $d < b$ 时, (c, d) 受控于 (a, b) , 从而 (c, d) 不是 $p[i]$ 中的跳跃点。除受控跳跃点外, $p[i+1] \cup q[i+1]$ 中的其它跳跃点均为 $p[i]$ 中的跳跃点。

- 由此可见, 在递归地由表 $p[i+1]$ 计算表 $p[i]$ 时, 可先由 $p[i+1]$ 计算出 $q[i+1]$, 然后合并表 $p[i+1]$ 和表 $q[i+1]$, 并清除其中的受控跳跃点得到表 $p[i]$ 。

$n=5, c=10, w=\{2, 2, 6, 5, 4\}, v=\{6, 3, 5, 4, 6\}$ 。

矩阵如下：0

0	6	6	6	6	6	6	6	6	6
0	6	6	9	9	9	9	9	9	9
0	6	6	9	9	9	9	11	11	14
0	6	6	9	9	9	10	11	13	14
0	6	6	9	9	12	12	15	15	15

跳跃点的计算过程如下：

初始时 $p[6]=\{(0, 0)\}$

因此， $q[6]=p[6] \oplus (w[5], v[5])=\{(4, 6)\}$

$p[5]=\{(0, 0), (4, 6)\}$

$q[5]=p[5] \oplus (w[4], v[4])=\{(5, 4), (9, 10)\}$

$p[5]$ 与 $q[5]$ 的并集 $p[5] \cup q[5]=\{(0, 0), (4, 6), (5, 4), (9, 10)\}$ 中跳跃点 $(5, 4)$ 受控于跳跃点 $(4, 6)$ 。

将受控跳跃点 $(5, 4)$ 清除后，得到 $p[4]=\{(0, 0), (4, 6), (9, 10)\}$

$q[4]=p[4] \oplus (6, 5)=\{(6, 5), (10, 11)\}$

$p[3]=\{(0, 0), (4, 6), (9, 10), (10, 11)\}$

$q[3]=p[3] \oplus (2, 3)=\{(2, 3), (6, 9)\}$

$p[2]=\{(0, 0), (2, 3), (4, 6), (6, 9), (9, 10), (10, 11)\}$

$q[2]=p[2] \oplus (2, 6)=\{(2, 6), (4, 9), (6, 12), (8, 15)\}$

$p[1]=\{(0, 0), (2, 6), (4, 9), (6, 12), (8, 15)\}$

$p[1]$ 的最后的那个跳跃点 $(8, 15)$ 即为所求的最优值， $m(n, C)=15$

5.第四章

流利的写代码还要会证明，分析算法的复杂度（注意包括排序， $n\log(n)$ ）后两个主要考实例（哈夫曼 最小生成树）

贪心代码简单，但要去证明。如果可以得到最优解，最优子结构（反证）、贪心性质证明

理解贪心算法概念

自顶向上。以迭代方式进行。

掌握贪心算法基本要素

首先假设存在一个最优解；然后证明存在一个以贪心选择开始的最优解；进一步，在做了贪心选择之后，子问题也是贪心选择获得的最优解。

最优子结构性质

一个问题的最优解包含其子问题的最优解。

贪心选择性质

所求问题的整体最优解可以通过一系列局部最优的选择（贪心选择）得到。用贪心选择出的第一个解与最优解完全

理解贪心和动态规划差异

贪心是自顶向上，通过迭代进行。动态规划是自底向上。

理解贪心算法的一般理论

贪心算法的一般框架

```
Greedy(C) //C是问题的输入集合即候选集合
{
    S={ }; //初始解集合为空集
    while (not solution(S)) //集合S没有构成问题的一个可行解
    {
        x=select(C); //在候选集合C中做贪心选择
        if constraint(S, x) //判断集合S中加入x后是否满足约束条件
        {
            S=S+{x};
            C=C-{x};
        }
    }
    return S;
}
```

活动安排

$n \log n$ （有排序，但排序算法不用写，设定一个sort函数调用就行）贪心策略，选最早的

```
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
    A[1]=true;
    int j=1;
    for (int i=2; i<=n; i++) {
        if (s[i]>=f[j]) { A[i]=true; j=i; }
        else A[i]=false;
    }
}
```

每次都选最早完成时间的相容活动加入A

如果已经按结束时间减序排列，就 $O(n)$ ；否则 $O(n \log n)$

背包问题

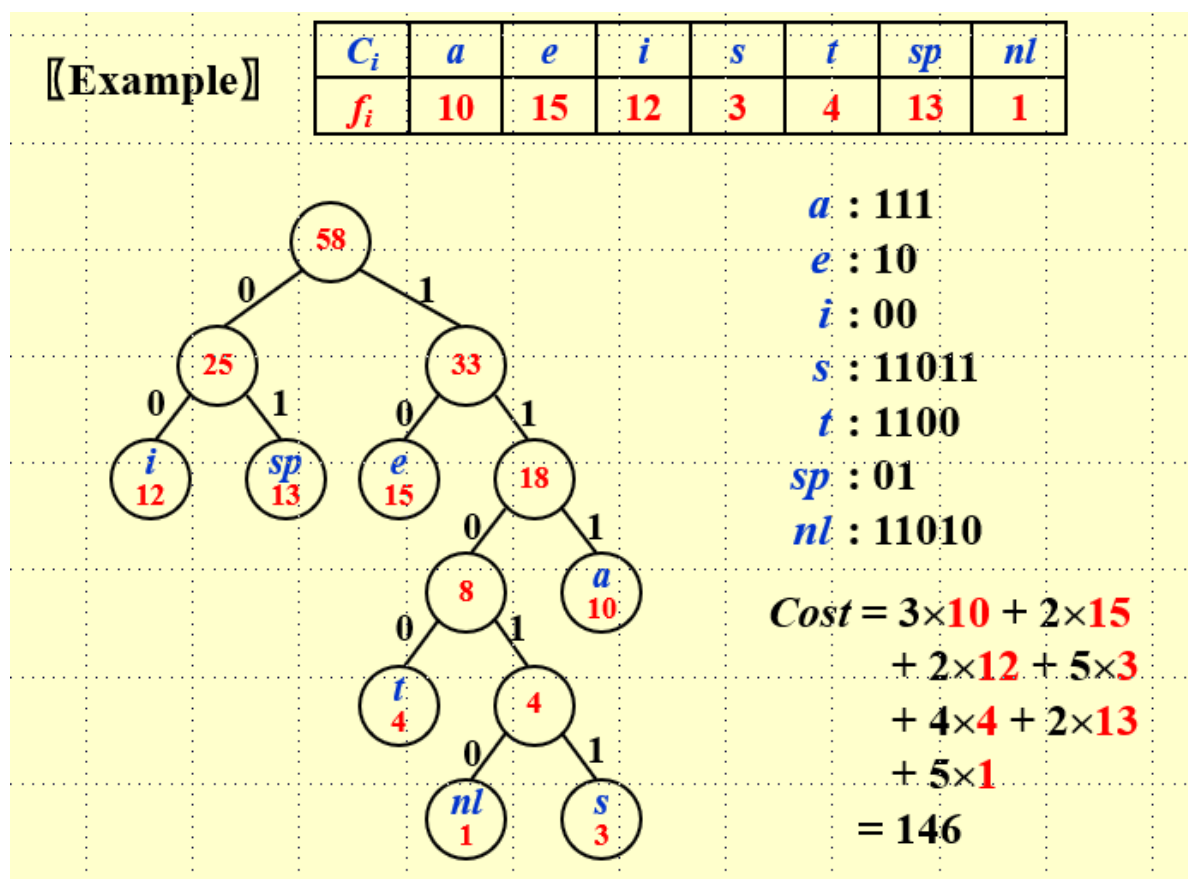
0-1背包——他不能得到最优解——无法保证始终能将背包装满。

不是0-1背包的话，重量和价值性价比排个序，物品——定被选中，然后选择2看看能不能装，装不下部分装

哈夫曼编码 很少考 太难了

编码、哈夫曼树要会画，写出来哈夫曼编码

左0右1，cost乘深度，先按大小去画树（二叉树），然后去填字符



最小生成树

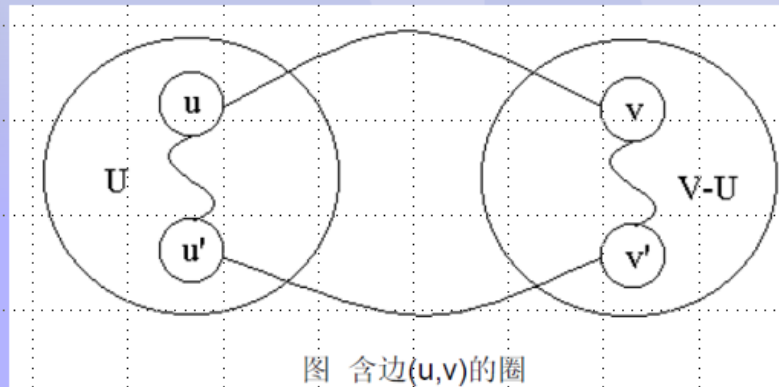
贪心证明可以不看，但一定要看mst边证明

设 $G = (V, E)$ 是无向连通带权图，即一个网络。E中每条边 (v, w) 的权为 $c[v][w]$ 。如果G的子图 G' 是一棵包含G的所有顶点的树，则称 G' 为G的生成树。生成树上各边权的总和称为该生成树的耗费。在G的所有生成树中，耗费最小的生成树称为G的最小生成树。

树包含了图中的所有点，不能用回路，与连通图不同。树能且仅能有 $n-1$ 条边。大于就是有回路，小于则不连通了。

设 $G=(V, E)$ 是连通带权图， U 是 V 的真子集。如果 $(u, v) \in E$ ，且 $u \in U$ ， $v \in V-U$ ，且在所有这样的边中， (u, v) 的权 $c[u][v]$ 最小，那么一定存在 G 的一棵最小生成树，它以 (u, v) 为其中一条边。这个性质有时也称为MST性质。

MST性质证明



假设 G 的任何一棵最小生成树都不含边 (u, v) 。将边 (u, v) 添加到 G 的一棵最小生成树 T 上，将产生含有边 (u, v) 的圈，并且在这个圈上有一条不同于 (u, v) 的边 (u', v') ，使得 $u' \in U, v' \in V-U$ ，如图所示。

将边 (u', v') 删去，得到 G 的另一棵生成树 T' 。由于 $c[u][v] \leq c[u'][v']$ ，所以 T' 的耗费 $\leq T$ 的耗费。于是 T' 是一棵含有边 (u, v) 的最小生成树，这与假设矛盾。

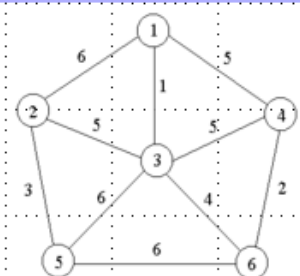
Prime 最近顶点 $O(n^2)$

2、Prim算法

设 $G=(V, E)$ 是连通带权图， $V=\{1, 2, \dots, n\}$ 。

构造 G 的最小生成树的Prim算法的基本思想是：首先置 $S=\{1\}$ ，然后，只要 S 是 V 的真子集，就作如下的贪心选择：选取满足条件 $i \in S$ ， $j \in V-S$ ，且 $c[i][j]$ 最小的边，将顶点 j 添加到 S 中。这个过程一直进行到 $S=V$ 时为止。

在这个过程中选取到的所有边恰好构成 G 的一棵最小生成树。



```

/
prim最小生成树

参数说明：
    G -- 邻接矩阵图
    start -- 从图中的第start个元素开始，生成最小树
/
void prim(Graph G, int start)
{
    int min,i,j,k,m,n,sum;
    int index=0;          // prim最小树的索引，即prims数组的索引
    char prims[MAX];      // prim最小树的结果数组
    int weights[MAX];     // 顶点间边的权值

    // prim最小生成树中第一个数是"图中第start个顶点"，因为是从start开始的。
    prims[index++] = G.vexs[start];

    // 初始化"顶点的权值数组"，
    // 将每个顶点的权值初始化为"第start个顶点"到"该顶点"的权值。
    for (i = 0; i < G.vexnum; i++)
        weights[i] = G.matrix[start][i];
    // 将第start个顶点的权值初始化为0。
    // 可以理解为"第start个顶点到它自身的距离为0"。
    weights[start] = 0;

    for (i = 0; i < G.vexnum; i++)
    {
        // 由于从start开始的，因此不需要再对第start个顶点进行处理。
        if(start == i)
            continue;

        j = 0;
        k = 0;
        min = INF;
        // 在未被加入到最小生成树的顶点中，找出权值最小的顶点。
        while (j < G.vexnum)
        {
            // 若weights[j]=0，意味着"第j个节点已经被排序过"(或者说已经加入了最小生成树
            // 中)。
            if (weights[j] != 0 && weights[j] < min)
            {
                min = weights[j];
                k = j;
            }
            j++;
        }

        // 经过上面的处理后，在未被加入到最小生成树的顶点中，权值最小的顶点是第k个顶点。
        // 将第k个顶点加入到最小生成树的结果数组中
        prims[index++] = G.vexs[k];
        // 将"第k个顶点的权值"标记为0，意味着第k个顶点已经排序过了(或者说已经加入了最小树结果
        // 中)。
        weights[k] = 0;
        // 当第k个顶点被加入到最小生成树的结果数组中之后，更新其它顶点的权值。
        for (j = 0 ; j < G.vexnum; j++)
        {
            // 当第j个节点没有被处理，并且需要更新时才被更新。
            if (weights[j] != 0 && G.matrix[k][j] < weights[j])

```

```

        weights[j] = G.matrix[k][j];
    }
}

// 计算最小生成树的权值
sum = 0;
for (i = 1; i < index; i++)
{
    min = INF;
    // 获取prims[i]在G中的位置
    n = get_position(G, prims[i]);
    // 在vexs[0...i]中, 找出到j的权值最小的顶点。
    for (j = 0; j < i; j++)
    {
        m = get_position(G, prims[j]);
        if (G.matrix[m][n] < min)
            min = G.matrix[m][n];
    }
    sum += min;
}
// 打印最小生成树
printf("PRIM(%c)=%d: ", G.vexs[start], sum);
for (i = 0; i < index; i++)
    printf("%c ", prims[i]);
printf("\n");
}

```

Kruskal 最短边 Oeloge

3、Kruskal 算法

Kruskal 算法构造G的最小生成树的**基本思想**是, 首先将G的n个顶点看成n个孤立的连通分支。将所有的边按权从小到大排序。然后从第一条边开始, 依边权递增的顺序查看每一条边, 并按下述方法连接2个不同的连通分支: 当查看到第k条边(v, w)时, 如果端点v和w分别是当前2个不同的连通分支T1和T2中的顶点时, 就用边(v, w)将T1和T2连接成一个连通分支, 然后继续查看第k+1条边; 如果端点v和w在当前的同一个连通分支中, 就直接再查看第k+1条边。这个过程一直进行到只剩下一个连通分支时为止。

当 $e = \Omega(n^2)$, prim好; 当 $e = o(n^2)$, Kruskal好

```

/
克鲁斯卡尔 (Kruskal) 最小生成树
/
void kruskal(Graph G)
{
    int i, m, n, p1, p2;

```

```

int length;
int index = 0;           // rets数组的索引
int vends[MAX]={0};      // 用于保存"已有最小生成树"中每个顶点在该最小树中的终点。
EData rets[MAX];         // 结果数组，保存kruskal最小生成树的边
EData edges;             // 图对应的所有边

// 获取"图中所有的边"
edges = get_edges(G);
// 将边按照"权"的大小进行排序(从小到大)
sorted_edges(edges, G.edgnum);

for (i=0; i<G.edgnum; i++)
{
    p1 = get_position(G, edges[i].start); // 获取第i条边的"起点"的序号
    p2 = get_position(G, edges[i].end);   // 获取第i条边的"终点"的序号

    m = get_end(vends, p1);               // 获取p1在"已有的最小生成树"中的终点
    n = get_end(vends, p2);               // 获取p2在"已有的最小生成树"中的终点
    // 如果m!=n, 意味着"边i"与"已经添加到最小生成树中的顶点"没有形成环路
    if (m != n)
    {
        vends[m] = n;                    // 设置m在"已有的最小生成树"中的终点为
n
        rets[index++] = edges[i];         // 保存结果
    }
}
free(edges);

// 统计并打印"kruskal最小生成树"的信息
length = 0;
for (i = 0; i < index; i++)
    length += rets[i].weight;
printf("Kruskal=%d: ", length);
for (i = 0; i < index; i++)
    printf("(%c,%c) ", rets[i].start, rets[i].end);
printf("\n");
}

```

单源最短路径 $O(n^2)$

给定带权有向图，每条边权值为非负实数。给定一顶点为源，要计算他到其他各顶点的最短路长度（各边权之和）。

初始时， S 中仅含有源。设 u 是 G 的某一个顶点，把从源到 u 且中间只经过 S 中顶点的路称为从源到 u 的特殊路径，并用数组 $dist$ 记录当前每个顶点所对应的最短特殊路径长度。Dijkstra算法每次从 $V-S$ 中取出具有最短特殊路径长度的顶点 u ，将 u 添加到 S 中，同时对数组 $dist$ 作必要的修改。一旦 S 包含了所有 V 中顶点， $dist$ 就记录了从源到所有其它顶点之间的最短路径长度。

贪心算法性质

- 每次的贪心选择是从V-S中选择具有最短的特殊路径的顶点U，从而确定从源到U的最短路径长度 $\text{Dist}[u]$ 。
- 证明：反证法
 - 假设从源s→u存在一个更好的路径，这条路径从源s出发，经过x，到达u。即
 - $D(s,x)+d(x,u) = d(s,u) < \text{dist}[u]$
 - 由于 $\text{Dist}[x] \leq d(s,x)$ 所以 $\text{dist}[x] < \text{dist}[u]$, 导致矛盾。

最优子结构性质

该性质描述为：如果 $S(i,j)=\{V_i \dots V_k \dots V_s \dots V_j\}$ 是从顶点i到j的最短路径，k和s是这条路径上的一个中间顶点，那么 $S(k,s)$ 必定是从k到s的最短路径。下面证明该性质的正确性。

假设 $S(i,j)=\{V_i \dots V_k \dots V_s \dots V_j\}$ 是从顶点i到j的最短路径，则有 $S(i,j)=S(i,k)+S(k,s)+S(s,j)$ 。而 $S(k,s)$ 不是从k到s的最短距离，那么必定存在另一条从k到s的最短路径 $S'(k,s)$ ，那么 $S'(i,j)=S(i,k)+S'(k,s)+S(s,j) < S(i,j)$ 。则与 $S(i,j)$ 是从i到j的最短路径相矛盾。因此该性质得证。

//4d5 贪心算法 单源最短路径问题

```
#include "stdafx.h"
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

const int N = 5;
const int M = 1000;
ifstream fin("4d5.txt");

template<class Type>
void Dijkstra(int n,int v,Type dist[],int prev[],Type c[][N+1]);
void Traceback(int v,int i,int prev[]); //输出最短路径 v源点, i终点

int main()
{
    int v = 1; //源点为1
    int dist[N+1],prev[N+1],c[N+1][N+1];

    cout<<"有向图权的矩阵为: "<<endl;
    for(int i=1; i<=N; i++)
    {
        for(int j=1; j<=N; j++)
        {
            fin>>c[i][j];
            cout<<c[i][j]<<" ";
        }
        cout<<endl;
    }

    Dijkstra(N,v,dist,prev,c);

    for(int i=2; i<=N; i++)
```



```

    {
        cout<<"源点1到点"<<i<<"的最短路径长度为:"<<dist[i]<<"，其路径为";
        Traceback(1,i,prev);
        cout<<endl;
    }

    return 0;
}

template<class Type>
void Dijkstra(int n,int v,Type dist[],int prev[],Type c[][N+1])
{
    bool s[N+1];
    for(int i=1; i<=n; i++)
    {
        dist[i] = c[v][i];//dist[i]表示当前从源到顶点i的最短特殊路径长度
        s[i] = false;

        if(dist[i] == M)
        {
            prev[i] = 0;//记录从源到顶点i的最短路径i的前一个顶点
        }
        else
        {
            prev[i] = v;
        }
    }

    dist[v] = 0;
    s[v] = true;

    for(int i=1; i<n; i++)
    {
        int temp = M;
        int u = v;//上一顶点

        //取出v-s中具有最短特殊路径长度的顶点u
        for(int j=1; j<=n; j++)
        {
            if((!s[j]) && (dist[j]<temp))
            {
                u = j;
                temp = dist[j];
            }
        }
        s[u] = true;

        //根据作出的贪心选择更新Dist值
        for(int j=1; j<=n; j++)
        {
            if((!s[j]) && (c[u][j]<M))
            {
                Type newdist = dist[u] + c[u][j];
                if(newdist < dist[j])
                {
                    dist[j] = newdist;
                    prev[j] = u;
                }
            }
        }
    }
}

```

```

    }
    }
}

//输出最短路径 v源点, i终点
void Traceback(int v,int i,int prev[])
{
    if(v == i)
    {
        cout<<i;
        return;
    }
    Traceback(v,prev[i],prev);
    cout<<"->"<<i;
}

```

最短路径证明认真学，求路径，写伪代码

6.第五章

解向量、解空间树、剪枝函数、剪枝后的解空间树、伪代码、复杂度
理解回溯法的深度优先搜索策略

- (1) 它在问题的解空间树中，按深度优先策略，从根节点出发搜索解空间树。
- (2) 算法搜索至解空间树的任一结点时，先判断该节点是否包含问题的解。
 - 如果肯定不包含，则跳过对以该结点为根的子树的搜索，逐层向其祖先节点回溯；
 - 否则，进入该子树，继续按深度优先策略搜索

回溯法要回溯到根且根节点的所有子树都被搜索遍才可以。他找到一个解就停止。

为了用回溯法求解一个具有n个输入的问题，一般情况下，将其可能的解表示为满足某个约束条件的等长向量 $X = (x_1, x_2, \dots, x_n)$ ，其中分量 x_i ($1 \leq i \leq n$)的取值范围是某个有限集合

$S_i = \{a_{i1}, a_{i2}, \dots, a_{in}\}$,所以有可能的解向量构成了问题的解空间。

回溯法从根结点出发，按照深度优先策略遍历解空间树，搜索满足约束条件的解。在搜索至树中任一结点时，先判断该结点对应的部分解是否满足约束条件，或者是否超出目标函数的界，也就是判断该结点是否包含问题的（最优）解，如果肯定不包含，则跳过对以该结点为根的子树的搜索，即所谓剪枝（Pruning）；否则，进入以该结点为根的子树，继续按照深度优先策略搜索。

剪枝可以利用题目自身约束条件，也可以利用筛选函数

自身约束：比如背包，已经超容量了就不用往下走了

筛选函数：比如售货员，已经超过之前求出的一个解了，就不用往下走了

- 如果解空间树中从根结点到叶结点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。而显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 内存空间。

掌握回溯法解题的算法框架

- (1) 如果 $X=(x_1, x_2, \dots, x_{i+1})$ 是问题的最终解，则输出这个解。如果问题只希望得到一个解，则结束搜索，否则继续搜索其他解；
- (2) 如果 $X=(x_1, x_2, \dots, x_{i+1})$ 是问题的部分解，则继续构造解向量的下一个分量；
- (3) 如果 $X=(x_1, x_2, \dots, x_{i+1})$ 既不是问题的部分解也不是问题的最终解，则存在下面两种情况：
 - ① 如果 $x_{i+1} = a_{i+1,k}$ 不是集合 S_{i+1} 的最后一个元素，则令 $x_{i+1} = a_{i+1,k+1}$ ，即选择 S_{i+1} 的下一个元素作为解向量 X 的第 $i+1$ 个分量；
 - ② 如果 $x_{i+1} = a_{i+1,k}$ 是集合 S_{i+1} 的最后一个元素，就回溯到 $X=(x_1, x_2, \dots, x_i)$ ，选择 S_i 的下一个元素作为解向量 X 的第 i 个分量，假设 $x_i = a_{ik}$ ，如果 a_{ik} 不是集合 S_i 的最后一个元素，则令 $x_i = a_{i,k+1}$ ；否则，就继续回溯到 $X=(x_1, x_2, \dots, x_{i-1})$ ；

回溯法的求解过程

递归回溯、迭代回溯、子集树算法框架、排列树算法框架

递归 (recursion)：递归常被用来描述以自相似方法重复事物的过程，在数学和计算机科学中，指的是在函数定义中使用函数自身的方法。（A调用A）

迭代 (iteration)：重复反馈过程的活动，每一次迭代的结果会作为下一次迭代的初始值。（A重复调用B）

递归回溯

```

void Backtrack(int t){
    if(t > n) output(x); //Output 记录或者输出可行解
    else{
        for( int i = f(n,t); i <= g(n,t); ++i){ //f(n,t)和g(n,t)表示在当前结点未搜索过的
            子树的起始编号和终止编号
            x[t] = h(i);
            if(constraint(t) && Bound(t)) Backtrack(t+1); //constraint和bound分别是
            约束函数和界限函数
        }
    }
}

```

迭代回溯

```

void IterativeBacktrack(void){
    int t = 1;
    while(t > 0){
        if(f(n,t) < g(n,t)){
            for(int i = f(n,t); i <= g(n,t); ++i){ //这个for 是遍历各个值的意思，实际中
            写成for循环会有逻辑错误
                x[t] = h(i);
                if(constraint(t) && bound(t)){
                    if(solution(t)) output(x); //solution 判断是否已经得到问题的解
                    else t++;
                }
                else t--;
            }
        }
    }
}

```

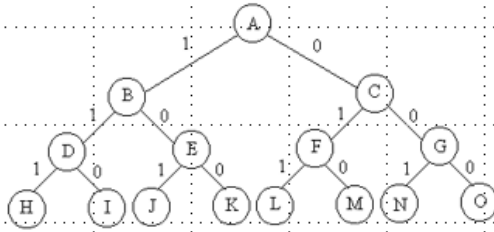
子集树 0/1背包

当所给的问题是从 n 个元素的集合 S 中找出满足某种性质的子集时，相应的解空间称为子集树

排列树 TSP

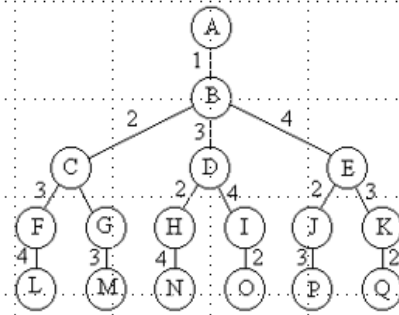
当所给的问题是确定 n 个元素满足某种性质的排列时，相应的解空间树成为排列树。

子集树与排列树



遍历子集树需 $O(2^n)$ 计算时间

```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=0;i<=1;i++) {
            x[t]=i;
            if (legal(t)) backtrack(t+1);
        }
}
```



遍历排列树需要 $O(n!)$ 计算时间

```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=t;i<=n;i++) {
            swap(x[t], x[i]);
            if (legal(t)) backtrack(t+1);
            swap(x[t], x[i]);
        }
}
```

轮船装载问题 $O2^n$

轮船第一种 不超载

两艘，尽量装满一艘，然后去装第二艘，这样就变成了一艘问题

子集树，特别像0/1背包问题

剪枝策略：左子树利用不能超载（左子树一定是选择该集装箱）右子树（不选择）用最优解选择，选择最大的去装载，将剩下的全装还没有bestw好，就没有必要再深入了

在类Loading中增加两个私有数据成员x和bestx。

- » X记录从根至当前结点的路径, $x[i]=0/1; [1\ 0\ 1\ 1\ 0]$
- » Bestx记录当前最优解，算法搜索到达一个叶子结点处，就修正bestx的值。

```
class Loading
{
    friend Type MaxLoading(Type [], Type int);
    //初始化操作
private:
    void Backtrack( int i); //对解空间搜索
    int n; //集装箱数
    *x, //当前解, 数组
    *bestx //当前最优解, 数组
    Type *w, //集装箱重量数组
    c, //第一艘轮船的载重量
    cw, //当前载重量
    bestw; //当前最优载重量
    r; //剩余集装箱的重量
};
```



```

template < class Type >
void Loading< Type > :: Backtrack( int i )
{
    if (i > n)
    {
        if (cw > bestw)
        {
            for(j = 1; j <= n; j++)
                bestx[j] = x[j];
            bestw = cw;
        }
        return;
    }
    //搜索子树
    r -= w[i];
    if ( cw + w[i] <= c)
    { //x[i] = 1; //二叉树的左孩子
        x[i] = 1;
        cw += w[i];
        Backtrack(i+1);
        cw -= w[i]; //回溯
    }
    if( cw+r > bestw)
    {
        x[i] = 0;
        Backtrack(i+1); //x[i] = 0; //二叉树的右孩子
    }
    r += w[i];
}

```

复杂度分析

由于bestx可能被更新 $O(2^n)$ 次，改进后算法的计算时间复杂度为 $O(n2^n)$ 。

数组x记录了解空间树中从根到当前扩展结点的路径，这些信息已包含了回溯法在回溯时所需信息。因此利用数组x所含信息，可将上述回溯法表示成非递归形式。由此可进一步省去 $O(n)$ 递归栈空间。

```

public static int maxLoading (int [] w,
int c, int [] bestx)
{ //迭代回溯法
    // 返回最优载重量及其相应解
    // 初始化根结点
    int i = 1; // 当前层
    int n = w.length - 1;
    int [] x = new int [n + 1]; // x[1:i-1]
    为当前路径
    int bestw = 0; // 当前最优载重量
    int cw = 0; // 当前载重量
    int r = 0; // 剩余集装箱重量
    for (int j = 1; j <= n; j++)
        r += w[j];
}

```

```

// 搜索子树
while (true){
    while (i <= n && cw + w[i] <= c)
    { // 进入左子树
        r -= w[i];
        cw += w[i];
        x[i] = 1;
        i++;
    }
    if (i > n)
    { // 到达叶结点
        for (int j = 1; j <= n; j++)
            bestx[j] = x[j];
        bestw = cw;
    }
}

```

迭代回溯

```

else { // 进入右子树
    r -= w[i];
    x[i] = 0;
    i++; }
while (cw + r <= bestw)
{ // 剪枝回溯
    i--;
    while (i > 0 && x[i] == 0)
    { // 从右子树返回
        r += w[i];
        i--; }
    if (i == 0) { delete [ ] x; return bestw; }
    // 进入右子树
    x[i] = 0;
    cw -= w[i];
    i++; }
}
}

```

作业调度问题 On!

定义的时间之和要好好理解，定义方式有点特殊，不要理解错

排列树

作业调度考虑两个因素，注意好时间，没超过最优解就不进入了。

还有个剪枝，有没有这条路径，课堂上没有详细讲，录音里面有

```

class Flowshop {
    friend Flow(int**, int, int []);
private:
    void Backtrack(int i);
    int **M, // 各作业所需的处理时间
    *x, // 当前作业调度
    *bestx, // 当前最优作业调度
    *f2, // 机器2完成处理时间
    f1, // 机器1完成处理时间
    f, // 完成时间和
    bestf, // 当前最优值
    n; // 作业数};

```

批处理作业调度问题



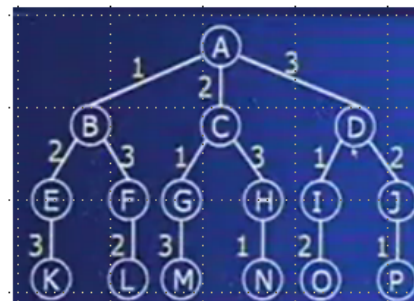
$M[1][1]$ = 作业1在机器1上工作的时间;
 $M[2][1]$ = 作业2在机器1上工作的时间;

```

void Flowshop::Backtrack(int i)
{
    if (i > n) {
        for (int j = 1; j <= n; j++)
            bestx[j] = x[j];
        bestf = f;
    }
    else
        for (int j = i; j <= n; j++) {
            f1 += M[x[j]][1];
            f2[i] = ((f2[i-1] > f1) ? f2[i-1] : f1) + M[x[j]][2];
            f += f2[i];
            if (f < bestf) {
                Swap(x[i], x[j]);
                Backtrack(i+1);
                Swap(x[i], x[j]);
            }
            f1 -= M[x[j]][1];
            f -= f2[i];
        }
}

```

t_{ij}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3



批处理作业调度问题



为什么swap?因为这是排列树, i 代表第几层, j 表示在第 i 层选择第几个作业。

TSP旅行售货员问题 On!

backtrack在最坏情况下可能需要更新最优解 $O(n-1)!$ ，每次更新bestx需要On，因此需要On!

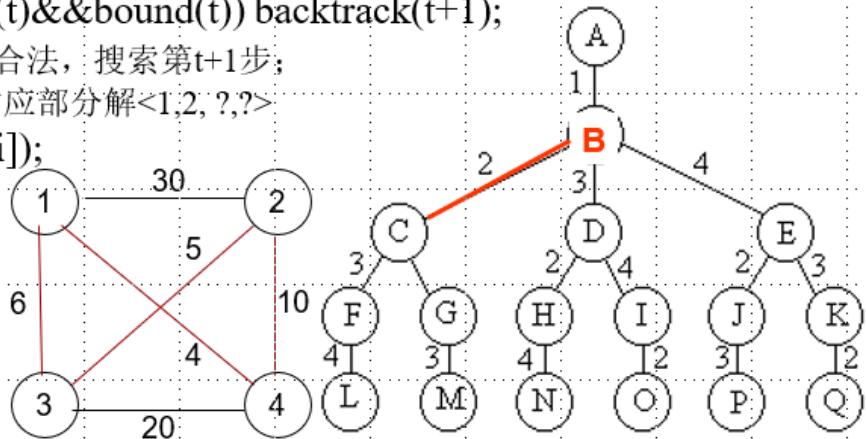
旅行收获 没有刚才的解好，没必要继续

排列树

剪枝：路径不存在或

往后走还没有往下已经超了最好旅费，就没有必要往后（利用问题本身最优解

void backtrack (int t)	// 已选择前t-1个城市， 部分解<x[1],x[2],...,x[t-1], ?, ?, ..., ?>;
{	本次考虑选定第t个城市， e.g. t=2, 对应于节点B、第2个 城市，当前部分解<1, ?, ?, ?>
if (t>n) output(x);	
else	
for (int i=t;i<=n;i++)	{//依次考虑选择后面的t、 t+1、 ..n步 所走城市.e.g. 第2、 3、 4步
swap(x[t], x[i]);	//初始时， x[i]=i. 将第t步设置为x[t]=i。 E.g. x[2]=2, 部分解< 1, i, ?, ?>, i=2,3,4
if (constraint(t)&&bound(t)) backtrack(t+1);	
//当前部分解合法， 搜索第t+1步； e.g. C结点, 对应部分解<1,2, ?, ?>	
swap(x[t], x[i]);	
}	
}	



bestx[1: n]: 记录最佳路径; cw: 当前部分路径的总长

BacktrackTSP(i)

```
1. if i=n then //已经搜索到叶节点, 已经选择了最后1个城市
   {
2.     if  $w(x[n-1], x[n]) \neq \infty$  and  $w(x[n], 1) \neq \infty$ 
           // 最后1个城市与前一城市相连、与第1个城市相连,
           组成1个满足条件的回路
3.     then //比较当前回路与以前最佳回路, 看当前回路
           是否更优
4.         if  $cw + w(x[n-1], x[n]) + w(x[n], 1) < bestw$ 
5.         then { //当前回路更优, 更新最优搜索结果
6.              $bestw = cw + w(x[n-1], x[n]) + w(x[n], 1)$ ;
7.             for j=1 to n do bestx[j]=x[j]
           }
   }
```

```
1. if i=n then //已经搜索到叶节点, 已经选择了最后1个城市
   {
   ...
7.     for j=1 to n do bestx[j]=x[j]
   }
8. else //如果搜索没有到叶节点, 当前得到的部分路径
       <1, x[2], ..., x[i-1], ?, ..., ?>
   {
9.     for j=i to n do //考察x[i]的各个可能取值
10.        if  $w(x[i-1], x[j]) \neq \infty$  and  $cw + w(x[i-1], x[j]) < bestw$ 
           // 向当前部分路径加入新城市x[j]后, x[j]与x[i-1]相
           连, 且扩展后的部分路径<1, x[2], ..., x[i-1], x[j], ..., ?>
           的成本小于当前最优回路的长度
11.        then //继续扩展、搜索次路径
```


9.	for j=i to n do //考察x[i]的各个可能取值
10.	if w(x[i-1], x[j]) $\neq \infty$ and cw + w(x[i-1], x[j]) < bestw // 向当前部分路径加入新城市x[j]后, x[j]与x[i-1]相 连, 且扩展后的部分路径<1, x[2],..., x[i-1], x[j], ..., ?> 的成本小于当前最优回路的长度
11.	then //继续扩展、搜索次路径
	{
12.	x[i] = x[j] //加入第i个城市
13.	cw = cw + w(x[i-1], x[i]) //更新扩展后的路径的代价
14.	BacktrackTSP(i+1) //递归搜索以x[i]为根的 后续子树
15.	cw = cw — w(x[i-1], x[i]) //搜索失败, 回溯, 回到 第9步, 为搜索x[i]的另一个取值x[j+1]准备
16.	x[i] = x[j]
	}

N皇后问题

最多考4皇后

解向量设计

解空间树 根是空棋盘, 每一层代表对每一行的选择

剪枝: 问题本身约束条件。行列号不能同, 不能在对角线

数目常见的有n!(排列树)、 a^n (子集树)和 n^n (可复选的排列树), 则回溯算法的**最坏时间复杂度**分别为:

- $O(p(n) * n!)$
- $O(p(n) * 2^n)$
- $O(p(n) * n^n)$

其中, $p(n)$ 表示生成一个节点所需的时间复杂度。

$p_n=n$

由于两个皇后不能位于同一列上，所以，解向量 X 必须满足**约束条件1**：

$$x_i \neq x_j$$

若两个皇后摆放的位置分别是 (i, x_i) 和 (j, x_j) ，在棋盘上斜率为-1的斜线上，满足条件 $i-j = x_i - x_j$ ，在棋盘上斜率为1的斜线上，满足条件 $i+j = x_i + x_j$ ，综合两种情况，由于两个皇后不能位于同一斜线上，所以，解向量 X 必须满足**约束条件2**：

$$|i - x_i| \neq |j - x_j|$$

四皇后为完全四叉树

四皇后问题的解空间树是一棵**完全4叉树**，树的根结点表示搜索的初始状态，从根结点到第2层结点对应皇后1在棋盘中第1行的可能摆放位置，从第2层结点到第3层结点对应皇后2在棋盘中第2行的可能摆放位置，依此类推。

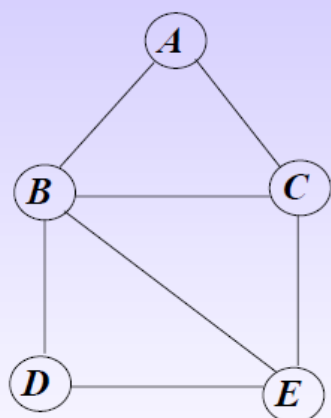
```
bool Queen::Place(int k)
{
    for (int j=1;j<k;j++)
        if ( (abs(k-j)==abs(x[j]-x[k])) || (x[j]==x[k]) ) return false;
    return true;
}

void Queen::Backtrack(int t)
{
    if (t>n) sum++;
    else
        for (int i=1;i<=n;i++) {
            x[t]=i;
            if (Place(t)) Backtrack(t+1);
        }
}
```

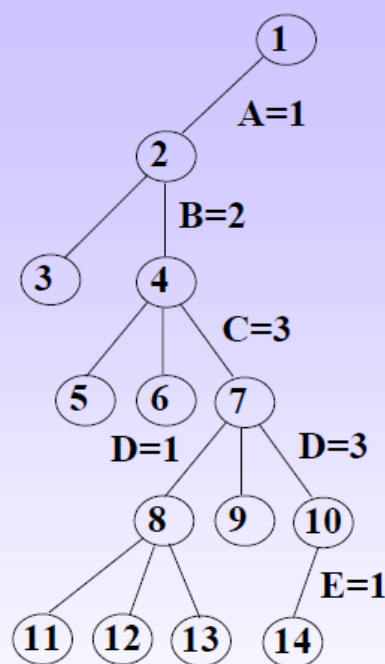
图的着色问题 Omn

给定无向连通图 G 和 m 种不同的颜色。用这些颜色为图 G 的各顶点着色，每个顶点着一种颜色。是否有一种着色法使 G 中每条边的2个顶点着不同颜色。这个问题是图的 m 可着色判定问题。若一个图最少需要 m 种颜色才能使图中每条边连接的2个顶点着不同颜色，则称这个数 m 为该图的色数。求一个图的色数 m 的问题称为图的 m 可着色优化问题。

例：判断下图是否是3可着色。



(a) 一个无向图



(b) 回溯法搜索空间

•解向量: $(x_1, x_2, \dots, x_n)$ 表示顶点 i 所着颜色 $x[i]$

•可行性约束函数: 顶点 i 与已着色的相邻顶点颜色不重复。

```

void Color::Backtrack(int t)
{
    if (t>n) {
        sum++;
        for (int i=1; i<=n; i++)
            cout << x[i] << ' ';
        cout << endl;
    }
    else
        for (int i=1;i<=m;i++) {
            x[t]=i;
            if (Ok(t)) Backtrack(t+1);
        }
}

bool Color::Ok(int k)
{
    // 检查颜色可用性
    for (int j=1;j<=n;j++)
        if ((a[k][j]==1)&&(x[j]==x[k])) return false;
    return true;
}

```

解向量设计：n个顶点 n维

剪枝条件：邻结矩阵，邻结就颜色不能相同